# Lab 3: Architecture Selection & Justification

*Problem Statement #10 — Student Club Event Ticketing & Budget Portal*

**Architecture Selection:** We chose Microservices Architecture for the Student Club Event Ticketing & Budget Portal System.

### Reason 1: Independent Scaling for Peak Loads

The system has two workloads with very different performance profiles: the QR-code ticket scanner must validate entries in under 100 ms during peak check-in rushes at events, while the multi-stage budget approval workflow (Faculty Coordinator → Finance Office → Dean) is comparatively slow and driven by human sign-off. Microservices let the Ticketing Service be deployed and scaled independently — e.g., with extra replicas during a large event — without touching or over-provisioning the Approval Workflow Service.

### Reason 2: Fault Isolation Across Independent Business Domains

Event proposals, budget approvals, ticketing, and notifications are naturally separate business capabilities owned by different stakeholders (club leads vs. finance/administration). If the Notification Service or Budget Management Service experiences downtime or a bug, the Ticketing Service can continue validating tickets at the door — a failure that would be far harder to contain in a single monolithic, layered application.

### Security Advantage

Each service enforces its own access control boundary at its API gateway. The Budget Management Service and Approval Workflow Service can require elevated, role-based authentication (Finance Office / Dean credentials) that is never exposed to the Club Portal or Ticketing Service, limiting the blast radius of a compromised client-facing component and preventing a breach of the public-facing kiosk from reaching financial data.

### Performance Benefit

Because the Ticketing Service is a small, independently deployed service with a narrow responsibility (QR generation and validation), it can be optimized and cached (e.g., in-memory ticket/session lookups) specifically to meet the sub-100 ms NFR at venue check-in, rather than sharing a request pipeline and database connection pool with slower, transaction-heavy approval and budgeting logic.

COMPONENT DIAGRAM:

